CAST 추가자료

정식화와 알고리즘 · 선행 정책 재현 절차

본편: CAST — 리전 용량 제약을 고려한 탄소 인지 시공간 워크로드 스케줄링

작성 2026-10-08

수록 목록

절	내용	본편에서 참조하는 곳
A.1	CAST 의 정식화와 알고리즘	3.2 절 · 3.3 절 · 3.4 절 · 3.5 절
A.2	재현 원칙과 공용 채점 규약	4.4 절
A.3	CASPER — 공개 코드의 MILP	4.4 절, 표 4
A.4	CICS — Virtual Capacity Curve	4.4 절, 표 4
A.5	Sukprasert — 공개 아티팩트의 탐색 알고리즘	4.4 절, 표 4
A.6	Caspian — 원문 정수 계획 정식화	4.4 절, 표 4
A.7	대응시키지 않은 정책과 그 이유	4.4 절
A.8	선행 연구의 평가 규모	5 절
A.9	지연 등급 분포를 바꾸어 재현한 결과	5 절
A.10	아티팩트 — 코드와 재현 절차	4.1 절

표 번호는 본편과 구분하기 위해 S 를 붙인다(표 S1 … 표 S18). 본편의 번호와 섞이지 않는다. 본편의 표·그림·식을 가리킬 때는 "본편 표 4"처럼 적는다.

A.1 CAST 의 정식화와 알고리즘

본편 3 장의 식 (1)~(15)와 Algorithm 1 을 한자리에 모은다. 식 번호는 본편과 같다. 각 식에는 한 줄 설명만 달고, 그 선택을 한 사유는 본편에 있다.

A.1.1 표기

표 S1. 기호와 값

기호	뜻	값 또는 범위
j ∈ J	작업	146,000 건
r ∈ R	리전	8 개
o_j	홈 리전	리전마다 18,250 건씩 균등
r(j)	실행 리전 (공간 이동의 산출)	—
s_j	제출 시각	1 년 = 8,760 시간에 분포
d_j	실행 시간	최소 1 초 · 중앙값 80.8 초 · 최대 6 시간
D_j	마감 시각	$s_j + L_j + d_j$
k_j	지연 등급 (작업이 신고)	1~5, 분포 3:1:1:2:3
L_j	지연 예산	k=1 12~24 시간 … k=5 1~5 초
τ_j	실행 시작 시각	결정 변수
x_{jr}	배정 이진 변수	0 또는 1
z_j	미배정 변수	실측 0 건
$C_r(t)$	실측 탄소집약도	gCO_2/kWh, 1 시간 해상도
Ĉ_r(t)	LSTM 예측	매 슬롯 새로 발행
ℓ_or	네트워크 지연	8×8 행렬, 최댓값 244 ms
cap_r	리전 용량	16 = ⌈baseline 최대 13 × 1.2⌉
η	여유율	0.8

기호	뜻	값 또는 범위
K_r	동시 실행 상한 ⌊η·cap_r⌋	12 (baseline 최대치의 92%)
$n_r(t)$ · $a_r(t)$	시각 t 의 동시 실행 수 · 남은 자리	$a_r = K_r - n_r$
H	예측 지평	24 시간
P	작업당 소비 전력	1 kW 상수
α	공간 이동의 탄소–지연 가중치 (식 7)	슬롯마다 무릎점으로 결정, 평균 0.508
β(k)	시간 이동의 탄소–지연 가중치 (식 11)	(6−k)/5 — 등급으로 고정

A.1.2 문제 정식화 — 식 (1)~(5)

(1) $\min \ \Sigma_{j\in J} \ P \cdot \int_{\tau_j}^{\tau_j+d_j} C_{r(j)}(t)\, dt$

총 배출량 최소화. 결정 변수는 배정 x_{jr} 과 실행 시각 τ_j 다.

(2) $\Sigma_{r\in R} x_{jr} = 1, \ \forall j \in J$

모든 작업이 정확히 한 리전에 배정된다. 미배정은 허용하지 않는다.

(3) $|\{ j : r(j)=r, \tau_j \le t < \tau_j+d_j \}| \le \lfloor \eta \cdot cap_r \rfloor, \ \forall r, \forall t$

동시 실행 상한. 우변을 A.1.5 에서 K_r 로 줄여 쓴다.

(4) $s_j \le \tau_j, \ \ \tau_j + d_j \le D_j, \ \forall j$

실행은 제출 이후에 시작하고 마감 이내에 끝난다.

(5) $(\tau_j - s_j) + \ell_{o\,j\,r(j)} \le L_j, \ \forall j$

지연 예산. 구현에서는 강제하지 않는다 — 지연 행렬 최댓값 244 ms 가 가장 짧은 지연 예산보다도 작아 수치상 구속되지 않는다.

식 (1)~(5)는 달성 목표이고, 실제로 푸는 것은 이를 두 단계로 쪼갠 A.1.3 과 A.1.5 다. 하나의 정수 계획으로 풀지 않은 것은 규모(이진 변수 2,800 만 개)와 온라인성(배정 시점에 이후 도착을 모른다) 때문이며, 두 단계의 해가 원 문제의 최적해라는 보장은 없다.

A.1.3 공간 이동 — 식 (6)~(9)

(6) $\tilde{C}_r = \hat{C}_r \,/\, \max_{r'} \hat{C}_{r'}, \quad \ell_{or} = \ell_{or} \,/\, 244$

두 축의 정규화. 탄소만 슬롯별 최댓값으로 나눈다.

(7) $\min \Sigma_j \Sigma_r \left(\alpha \cdot \tilde{C}_r + (1-\alpha) \cdot \ell_{oj\,r} \right) \cdot x_{jr} + P_{slack} \cdot \Sigma_j z_j$

슬롯 배정의 목적함수. P_{slack} = 1,000 은 어떤 배정 비용보다 커서 배정이 가능한 한 미배정이 선택되지 않는다.

(8) $\Sigma_r x_{jr} + z_j = 1 \;\; \forall j, \quad \Sigma_j x_{jr} \le a_r \;\; \forall r$

배정 유일성과 리전별 잔여 자리. 여기의 a_r 은 공간 이동 자신의 장부이며, 시간 이동의 $a_r(t)$ 와 이름만 같고 서로 인계되지 않는다.

(9) $\alpha^* = \arg\min_\alpha \sqrt{\left(\tilde{\lambda}(\alpha)^2 + \tilde{\mu}(\alpha)^2 \right)}$

슬롯마다 α 를 정하는 규칙. 후보 11 개(0~1, 0.1 간격)의 배정을 각각 풀어 정규화 평균 지연 $\tilde{\lambda}$ 와 정규화 예상 배출 $\tilde{\mu}$ 를 얻고, 원점에 가장 가까운 점을 택한다. 풀이기는 PuLP · CBC.

1 년치 배정 결과

표 S2. 리전 간 배정 행렬 (α=auto, 건수, 합 146,000)

홈 ₩ 실행	캘리	텍사스	뉴욕	프랑스	독일	한국	인도	일본
캘리포니아	17,916	69	66	172	27	0	0	0
텍사스	8,282	4,850	1,587	3,507	24	0	0	0
뉴욕	539	168	12,018	5,108	417	0	0	0
프랑스	25	2	53	17,982	188	0	0	0
독일	190	7	433	13,438	4,182	0	0	0
한국	2,943	54	59	658	62	11,828	0	2,646
인도	430	6	322	12,955	1,137	206	2,742	452
일본	6,418	154	60	358	28	489	0	10,743
받은 수	36,743	5,310	14,598	54,178	6,065	12,523	2,742	13,841

프랑스와 캘리포니아가 62.3%를 받는다. 다만 표에서 또렷한 것은 받는 쪽이 아니라 보내는 쪽이다 — 독일은 자기 작업의 73.6%를, 인도는 71.0%를 프랑스로 보내고 홈에 각각 22.9%와 15.0%만 남긴다.

인도가 실행 리전으로 1.9%밖에 받지 않는 것은 인도에 작업이 없어서가 아니라 인도의 작업이 나가기 때문이다. baseline 은 대각선만 채워진 단위행렬이다(비대각 합 0 확인).

A.1.4 시간 이동의 점수 — 식 (10)~(11)

실행 가능 윈도우는 $[s_j,\ D_j - d_j]$ 이고 후보는 그 안의 1 시간 슬롯이다.

(10) $\bar{C}(t) = (\hat{C}(t) - C_{min})/(C_{max} - C_{min}),\ \ \bar{D}(t) = (t - t_{early})/(t_{late} - t_{early})$

탄소 비용과 지연 비용을 후보 집합 안에서 0–1 로 정규화한다. 분모가 0 이면 해당 항을 0 으로 둔다.

(11) $\text{score}(t) = \beta(k)\cdot\bar{C}(t) + (1-\beta(k))\cdot\bar{D}(t),\ \ \beta(k) = (6 - k)/5$

가중합. 가중치 β(k) 는 작업이 신고한 지연 등급의 함수이며 k=1 에서 1.0, k=5 에서 0.2 다. 공간 이동의 α 와 달리 자료로 정하지 않고 등급으로 고정한다.

A.1.5 용량 인지 온라인 배치 — 식 (12)~(14)와 Algorithm 1

(12) $n_r(t) = |\{ j : r(j)=r, \tau_j \le t < \tau_j + d_j \}|,\ \ a_r(t) = K_r - n_r(t)$

식 (3)을 남은 자리 형태로 다시 쓴 것. 장부는 이 단계가 시작할 때 비어 있고 이 단계가 확정한 τ_j 로만 채워진다.

$$\text{feasible}(j, t') \iff \min a_{r(j)}(u) \ge 1,\ \ u \in [t', t'+d_j)$$

시작 시점만이 아니라 실행이 끝날 때까지 매 순간 자리가 있어야 한다.

(13) $Q(t) = \{ j : s_j \le t,\ \tau_j \text{ 미정} \},$

$P_r(t) = \{ j \in Q(t) : r(j)=r, \text{argmin}_{t'} \text{score}_j(t') = \max(t, s_j) \}$

지금이 남은 탐색 구간 안에서 최선인 작업들. t′ 는 남은 실행 가능 윈도우 $W_j(t) = [\max(t,\ s_j),\ \min(D_j - d_j,\ t + H)]$ 이고, 점수는 매 슬롯 새로 발행된 예측으로 다시 잰다.

(14) $u_j(t) = (D_j - d_j) - t, \quad F_r(t) = \{ j \in Q(t) : r(j)=r, u_j(t) < 1 \},$

$R_r(t) = P_r(t) \setminus F_r(t)$ 를 $u_j(t)$ 오름차순 정렬,

$S_r(t) = F_r(t) \cup R_r(t)[\ 1 \ldots \max(a_r(t) - |F_r(t)|, 0)\ ]$

남은 여유가 적은 순(EDF)으로 자리만큼만 채운다. $F_r(t)$ 를 후보 $P_r(t)$ 가 아니라 대기 집합 Q(t) 전체에서 고르는 것이 요점이다 — P 안에서 고르면 '지금이 최선이 아닌' 마감 임박 작업이 후보에서 빠져 무한정 밀린다.

Algorithm 1

오른쪽 주석은 본편에 없는 구현 주의사항이다.

```
Algorithm 1  슬롯 단위 롤링 재평가

Input:  K_r, Ĉ_r^(t)(·), H, 작업 집합        Output: 모든 j 에 대한 τ_j

 1: Q ← ∅
 2: for t ← 0 to t_max do                    ▷ t_max = 8,760 + 2
 3:   for each j with ⌊s_j⌋ = t do
 4:     if ⌊D_j − d_j⌋ ≤ ⌊s_j⌋ then τ_j ← s_j  ▷ ⌊ ⌋ 없이 비교하면 영영 거짓
 5:     else Q ← Q ∪ { j }
 6:   for each r ∈ R do
 7:     a_r(t) ← K_r − n_r(t)                 ▷ 지연 pop 은 그 슬롯의 t 로만
 8:     F ← { j ∈ Q : r(j)=r, u_j(t) < 1 }    ▷ P 가 아니라 Q 전체에서
 9:     P ← ∅
10:     for each j ∈ Q with r(j)=r, j ∉ F do
11:       W_j(t) ← 실행 가능 윈도우
12:       if argmin_{t′∈W_j(t)} score_j(t′)
               = max(t, s_j) then            ▷ offset 하한은 항상 0
13:         P ← P ∪ { j }
14:     sort P by u_j(t) ascending
15:     S ← F ∪ P[ 1 … max(a_r(t) − |F|, 0) ] ▷ 위반이 정의상 0 인 경로
16:     for each j ∈ S do
17:       τ_j ← max(t, s_j);  Q ← Q \ { j }   ▷ 장부는 여기서만 채워진다
18:   if Q = ∅ and 남은 도착 없음 then break
```

4 번 줄과 12 번 줄의 주석은 실제로 겪은 버그다. $D_j - d_j$ 와 s_j 는 둘 다 분수 시각이라 바닥 함수 없이 비교하면 4 번 줄이 영영 거짓이 되어 즉시 실행 우회가 무용지물이 되고, 12 번 줄에서 후보 offset 의 하한을 $\lceil s_j - t \rceil$ 로 두면 '지금'이 후보에서 통째로 빠진다. 구현은 scheduler/capacity.py 의 run_rolling 이다.

보장

무한 연기 없음 — $u_j(t)$ 는 슬롯마다 정확히 1 씩 줄고 시작 값이 유한하므로 유한한 슬롯 안에 $F_r(t)$ 에 편입된다. 마감 위반 없음 — 후보 슬롯이 애초에 $D_j - d_j$ 이하로만 잡힌다. 두 보장은 독립이며, A.9 에서

용량을 조여 위반이 28,828 건까지 늘어나는 구간에서도 마감 위반이 0 인 것이 그 실측 확인이다. 슬롯당 비용은 대기 작업 수와 H 의 곱에 비례하고, 146,000 건 전체가 약 7.6 초에 끝난다.

상한을 넘기는 두 경로

4 번 줄의 즉시 실행은 용량을 확인하지 않는다 — 옮길 자리가 없는 작업을 막으면 마감을 넘기는 수밖에 없기 때문이다. 146,000 건 중 86,292 건(59.1%)이 이 경로로 가며, 지연 예산이 슬롯 길이 1 시간보다 짧은 등급 3·4·5 가 대부분이다. 상한 위반 227 건은 전부 여기와 마감 직전 강제 편입에서 나온다.

표 S3. 상한 위반 227 건의 경로별 분해

경로	의사코드	건수	왜 막을 수 없나
마감 직전 강제 편입 (F)	8·15 번 줄	106	이번 슬롯이 마지막 기회
즉시 실행	4 번 줄	121	정수 격자에 다른 후보가 없음
용량 확인 후 배정 (P)	15 번 줄	0	남은 자리만큼만 자르므로 정의상 0

표 S4. 지연 등급별 시간 이동 절감 기여

등급	작업 수	지연 예산	절감량 (kg)	비중 (%)
k=1	43,830	12~24 시간	1,692.8	93.79
k=2	14,458	2~8 시간	111.9	6.20
k=3	14,716	2~10 분	0.1	0.01
k=4	29,244	15~60 초	0.0	0.00
k=5	43,752	1~5 초	0.0	0.00

등급 3 이하가 0.00%인 것은 알고리즘의 한계가 아니라 슬롯 길이의 결과다. 이 셋은 지연 예산이 1 시간보다 짧아 정수 슬롯 격자 위에 옮길 자리가 없으며, 전체의 60.1%인 87,712 건이다.

A.1.6 탄소 회계 — 식 (15)

$$(15)\quad \mathrm{E_j} = \mathrm{P} \cdot \int_{\tau_j}^{\tau_j + d_j} \mathrm{C_{r(j)}}(t)\, dt$$

실측 계열이 1 시간 해상도이므로 슬롯 평균으로 근사한다.

$$E_j \approx P \cdot d_j \cdot \mathrm{mean}(\, C_r[\, a \,\ldots\, a+n-1 \,]\,), \quad a = \lfloor \tau_j \rfloor, \quad n = \max(1, \lceil d_j \rceil)$$

timeshift.mean_carbon 과 capacity._Windows.actual_mean 이 이 계산이며, 본편 표 3 의 다섯 행과 표 4 의 선행 정책이 모두 이 함수를 통과한다. 기준도 예외가 아니다 — 29,225.6 kg 은 로드밸런서의 baseline 산출물(적분 회계로 29,182.6 kg)이 아니라 scheduler 의 simple_lb_immediate 실행값이다. 결정은 예측값으로, 회계는 언제나 실측값으로 한다.

슬롯 평균이 근사이므로 같은 스케줄을 겹침 길이로 가중한 정확 적분으로 다시 채점했다. 작업 146,000 건 전수이며 스케줄은 손대지 않았다.

표 S5. 두 회계로 다시 채점한 결과 (본편 표 3 · 표 4)

대상	구현 (슬롯 평균)	정확 적분	절감률 차
공간만	12,609.8 kg	12,592.4 kg	+0.06%p
용량 무시	9,958.2 kg	9,989.7 kg	−0.11%p
CAST	10,805.0 kg	10,805.1 kg	0.00%p
CASPER (CAP, α=0.9)	1,379.1 kg	1,380.7 kg	−0.006%p
CICS (λp/λe = 0)	25,203.4 kg	25,276.8 kg	−0.251%p
Sukprasert 시간 단독	9,583.6 kg	9,571.8 kg	+0.041%p
Sukprasert 공간+시간	990.8 kg	990.6 kg	+0.001%p
Caspian (분수)	20,308.4 kg	20,335.4 kg	−0.092%p

헤드라인 CAST 는 두 회계에서 같은 값이고, 나머지도 0.3% 안이며 순위가 바뀌지 않는다. 차이의 방향은 리전의 성질로 설명된다 — 슬롯 평균과 정확 적분의 차이는 실행 구간 안에서 곡선이 얼마나 움직였는지에만 좌우되는데, 프랑스의 시간당 변동폭은 1.4 gCO_2/kWh 로 나머지 일곱(4.0~16.3)의 3 분의 1 에서 12 분의 1 이다. 차이가 가장 작은 정책이 거의 모든 작업을 프랑스에 모으는 Sukprasert 공간+시간인 것은 그 때문이다. 용량·마감 위반 건수는 회계와 무관하므로 두 방식에서 완전히 같다.

A.1.7 재현에 필요한 파라미터

표 S1 이 식에 나오는 기호를 다루므로, 여기서는 식에 안 나오지만 값을 모르면 재현이 안 되는 것만 모은다. 난수는 작업 자료를 만들 때만 쓰이고 스케줄링은 결정적이다.

표 S6. 식에 안 나오는 재현 파라미터

항목	값
슬롯 길이 · 평가 기간	1 시간(3,600 초) · 8,760 시간
α 후보 격자 · 풀이기	0.0~1.0, 0.1 간격 11 개 · PuLP CBC
미배정 페널티 P_{slack}	1,000
용량 계수	1.2 (cap_r = [baseline 최대 × 1.2])
LSTM	입력 168 시간 · 은닉 64 · 2 층 · dropout 0.2 · 피처 13(공통 10 + 기상 3)

지연 등급 분포는 고정된 가정이며 결과를 크게 좌우한다. 분포를 바꾸어 전체 파이프라인을 다시 돌린 결과가 A.9 다.

A.1.8 순진한 하한 대조군의 절차

표 S7. 순진한 하한 대조군의 리전별 분해 (본편 4.3 절) — 손실 편익은 kg, 건당 손실은 g

리전	최대 동시	초과 시간	되돌린 작업	손실 편익	건당 손실
캘리포니아	41	866.9	16,449	1,805.1	109.74
프랑스	41	622.4	13,258	54.3	4.09
나머지 6 개 리전	6~22	29.2 이하	1,543	55.4	—

본편 4.3 절의 대조군(순진한 하한)은 배치 시점에 용량을 전혀 모르는 스케줄에서 출발해 끝난 뒤에 초과를 없앤다. 일부러 단순하게 둔 절차이며, 용량을 아는 스케줄러보다 항상 같거나 나쁘므로 그 결과가 '아무것도 하지 않았을 때 감수할 손해'의 하한이 된다. 리전마다 독립으로 다음을 초과가 사라질 때까지 반복한다.

1. 그 리전 작업들의 (시작, 종료)로 연속 시각 스윕을 돌려 동시 실행 수가 K_r 을 넘는 구간을 찾는다.
2. 아직 되돌리지 않은 작업 중 그 구간과 겹치는 것을 전부 즉시 실행(τ_j ← s_j)으로 되돌린다.
3. 새로 되돌릴 것이 없으면 멈춘다.

되돌린 작업을 다시 배치하지는 않는다 — 대조군을 일부러 단순하게 둔 지점이 여기다. 1 년 전체에서 31,250 건(21.4%)이 되돌려지며 프랑스는 5 회, 캘리포니아는 3 회 만에 수렴한다. 회계는 앞 절의 슬롯 평균과 같은 함수다.

A.1.9 실행 시간

공간 이동의 무릎점 탐색(슬롯당 α 후보 11 개의 정수 계획)이 슬롯당 평균 174 ms, 최악 890 ms 이며 1 년 전체로 약 25 분이다. 시간 이동의 롤링 재평가는 146,000 건 전체가 약 7.6 초에 끝난다(단일 실행, 자료 적재 제외).

A.2 재현 원칙과 공용 채점 규약

본편 4.4 절의 비교표는 선행 연구의 정책을 개념적으로 흉내 낸 것이 아니라, 공개된 구현 코드와 원문 수식을 그대로 옮겨 본 연구와 동일한 데이터·동일한 채점 함수로 실행한 결과다. 그 재현이 정당한지는 결국 "무엇을 그대로 가져왔고 무엇을 바꾸었는가"에 달려 있는데, 이를 본편에 모두 적으면 분량이 감당되지 않는다. 이어지는 A.3~A.6 이 그 내역을 정책별로 남긴다.

개념적 대응이 왜 위험한지는 작업 도중 실제로 드러났다. 본 연구는 처음에 CASPER 의 가중치 α 를 "탄소와 지연의 절충"으로 읽었으나, 원문 Table 1 은 α 를 "서버 대수 대비 탄소집약도의 정규화 가중치"로 정의한다. 지연은 목적함수가 아니라 별도의 하드 제약이다. 같은 기호가 전혀 다른 것을 절충하고 있었다. 수식을 직접 옮기지 않았다면 이 차이는 표 안으로 조용히 들어갔을 것이다.

공용 채점 규약

모든 재현 구현은 "어느 작업을 어느 리전에서 언제 시작하는가"만 결정한다. 배출량은 구현이 직접 계산하지 않고, 전부 하나의 채점 함수를 통과한다. 이 함수는 본편 3.5 절 식 (15)의 회계를 쓴다 — 실행

구간에 대한 실측 탄소집약도의 적분이며, 실측 계열이 1 시간 해상도이므로 슬롯 평균으로 근사한다. 정확한 형태는 A.1.6 에 있다.

이 규약을 둔 이유는 실제 사고가 있었기 때문이다. 로드밸런서 산출물에 저장된 배출량 열을 그대로 더하면 기준 시나리오가 29,182.6 kg 이 되는데, 본편 표 3 의 기준은 29,225.6 kg 이다. 스케줄은 같고 회계만 다르다 — 전자는 로드밸런서가 실행 구간을 겹침 길이로 가중해 정확히 적분한 값이고, 후자는 본편이 쓰는 슬롯 평균 근사다. 차이는 0.147%로, A.1.6 의 표 S5 가 다른 단계에서 잰 차이(0.001~0.315%)와 같은 크기다. 구현마다 자기 회계를 들고 오면 이런 차이가 표 안에서 서로 다른 잣대가 된다. 채점 함수는 기준 시나리오를 29,225.6 kg 으로 정확히 재현함을 확인한 뒤 사용하였다.

용량 상한 위반 건수 역시 새로 정의하지 않고 본편 4.3 절이 쓰는 함수를 그대로 호출한다. 같은 스케줄을 자체 구현으로 세었을 때 127 건, 본편 정의로 세었을 때 227 건으로 갈렸다 — 같은 시각에 시작한 작업들이 서로를 "이미 실행 중"으로 보는지에서 갈리는 차이다. 본 연구가 227 쪽을 표준으로 삼는 까닭은 슬롯이 정수 시각이어서 같은 tick 에 여러 작업이 함께 시작하는 일이 흔하고, 그때 서로를 세지 않으면 실제 동시 점유를 과소계상하기 때문이다. 정의가 둘이면 표가 갈라진다.

A.3 CASPER — 공개 코드의 MILP

출처는 저자들이 공개한 저장소(github.com/carbonfirst/casper)의 CAP/milp_scheduler.py 다. 논문이 서술하는 정식화가 아니라 실제로 실행되는 코드를 옮겼다.

정식화

```
목적함수   min  α·(1/(max C·Σ r))·Σ x_ij·C_j  +  (1−α)·(1/S_max)·Σ s_i

지연 제약   x_ij·(ℓ_ij − L_max) ≤ 0        L_max = 500 ms   (하드)

용량 제약   Σ_i x_ij ≤ s_j · κ              κ = 10 요청/서버

수요 제약   Σ_j x_ij = r_i                   (전량 배정)

서버 상한   Σ_i s_i ≤ 100·|R|,  s_i ≤ 100    x, s ∈ 정수
```

여기서 C_j는 리전 j 의 탄소집약도, ℓ_ij 는 리전 i 에서 j 로의 지연, r_i는 리전 i 에서 발생한 요청 수, s_i는 리전 i 에 띄울 서버 수다. 두 항 앞의 계수는 코드가 쓰는 정규화 상수를 그대로 옮긴 것이다. α 는 탄소와 지연이 아니라 탄소와 서버 대수를 절충한다.

본 재현은 코드에 기록된 값 α=0.9 를 사용하였다. 원 논문 본문은 α=0.5 로 기술하나, 공개 코드에서 0.5 는 비교군인 지연 최소화 스케줄러의 값이며 탄소 스케줄러에는 0.9 가 적혀 있다. 두 스케줄러는 가중치만 다른 것이 아니라 목적함수의 첫 항 자체가 다르다 — 한쪽은 탄소집약도를, 다른 쪽은 지연을 싣는다. 재현은 실행되는 코드를 따랐다.

적용하며 내린 판단

CASPER 는 웹 요청률을 다루고 본 연구는 배치 작업을 다룬다. 매 시간 슬롯에서 그 시각 제출된 작업 수를 리전별로 세어 r_i로 두었다. CASPER 는 실행 시각을 옮기지 않으므로 시작 시각은 제출 시각이다. 원문 파라미터(L_{max}=500 ms, κ=10, 리전당 서버 100 대)는 본 연구 환경에 맞추어 조정하지 않고 그대로 두었다.

두 제약이 한 번도 활성화되지 않는다

이 결정은 결과 해석에 결정적이다. 본 연구의 리전 간 실측 지연은 최대 244 ms 이므로 L_{max}=500 ms 제약은 x 의 값과 무관하게 항상 성립한다. 또 시간당 전체 요청 수는 최대 38 건인데 리전 하나의 수용량은 100×10=1,000 건이므로, 한 시간의 전체 수요가 한 리전에 모두 몰려도 26 배의 여유가 남는다. 두 제약 모두 표본 관측이 아니라 최댓값 비교로 비활성이 확정된다.

그 결과 CASPER 의 해는 매 시간 모든 작업을 그 시각 가장 깨끗한 리전 하나로 보내는 것이 된다. 실제로 임의의 한 시각을 열어 보면 8 개 리전에서 발생한 요청 전부의 목적지가 프랑스 하나였다. 이는 구현 오류가 아니라 제약이 없는 최적화의 정상적인 귀결이다.

표 S8. CASPER 두 스케줄러를 본 연구 데이터에 적용한 결과

정책	총배출(kg)	절감률	평균 지연	마감 위반	용량 위반
CASPER 탄소 스케줄러 (α=0.9)	1,379.1	95.28%	120.4 ms	0	69,310
CASPER 지연 비교군 (α=0.5)	29,225.6	0.00%	0.0 ms	0	1
(참고) 본 연구 CAST	10,805.0	63.03%	36.2 ms	0	227

지연 비교군이 기준 시나리오와 정확히 같은 값을 낸 것은 구현이 옳다는 방증이다. 탄소를 전혀 보지 않고 지연만 최소화하면 어떤 작업도 옮기지 않는 것이 최적이므로, 홈 리전 즉시 실행과 같아지는 것이 맞다.

탄소 스케줄러의 1,379.1 kg 은 본 연구가 스스로에게 부과한 용량 상한을 CASPER 에는 적용하지 않았을 때의 값이다. 같은 상한으로 다시 세면 배정의 47.5%가 위반이다. CASPER 원문에 그러한 상한이 없으므로 임의로 조이지 않았으나, 이 숫자가 낮은 이유가 더 나은 정책이어서가 아니라 더 느슨한 용량 전제 때문이라는 점은 함께 읽어야 한다.

A.4 CICS — Virtual Capacity Curve

출처는 Radovanović 등의 논문 3 절 C 항, 식 (4)와 그 뒤의 제약식이다. 공개 코드가 없어 수식을 직접 옮겼다.

```
목적함수   min  λe·Σ_{c,h} η_c(h)·( Pow_c + π_c·δ(c,h)·τ_c/24 )  +  λp·Σ_c y_c
일일 보존   Σ_h δ(c,h) = 0      ∀c
첨두 정의   y_c ≥ (1+δ(c,h))·τ_c/24     ∀h
```

δ(c,h)는 클러스터 c 의 h 시각 유연 부하가 그 날 시간 평균에서 벗어나는 정도이고, y_c는 그 날의 첨두 상한이다. 일일 보존 제약이 CICS 의 핵심이다 — 하루 총량은 그대로 두고 시간별 모양만 바꾸므로, 모든 유연 작업이 그 날 안에 끝난다.

적용하며 내린 판단

Pow 항은 δ 에 무관한 상수이므로 최적해가 달라지지 않아 제외하였다. π 는 전력 민감도 계수인데 본 연구의 회계는 작업당 고정 전력을 쓰므로 상수가 된다. CICS 는 CPU 사용량을 다루고 본 연구는 개별 작업을 다루므로, 그 날 그 리전에 도착한 작업 수를 τ_c로 두고 곡선을 "그 시각에 시작을 허용하는 작업 수"로 읽었다. 이렇게 하면 일일 보존 제약이 "하루 총 허용량 = 그 날 도착한 작업 수"가 되어 원문의 뜻과 일치한다.

λp 와 λe 의 비를 알 수 없으므로 하나를 고르지 않고 0 에서 5 까지 쓸었다. 결과는 25,203.4 kg 에서 25,202.4 kg 사이로 사실상 움직이지 않았다. 본 연구의 작업은 길이가 짧고 마감이 촘촘해 곡선을 평탄하게 만들 여지 자체가 적기 때문이다.

CICS 에는 작업별 마감이 없고 "하루 안에 끝난다"만 있다. 본 연구의 작업에는 마감이 있으므로 대기열에서 마감이 임박한 순으로 뽑되, 이번 슬롯이 마지막 기회인 작업은 곡선의 상한을 넘겨 내보냈다. 이 예외가 37,594 건, 곧 전체의 26%에 해당한다. 이는 CICS 의 결함이 아니라 전제가 다른 데서 오는 차이다.

표 S9. CICS 와 본 연구를 같은 축에서 비교한 결과

정책 (홈 리전 고정, 시간 축만)	총배출(kg)	절감률	마감 위반	용량 위반
CICS Virtual Capacity Curve	25,203.4	13.76%	0	0
본 연구 시간만	23,789.1	18.60%	0	24

같은 축에서 본 연구가 4.84%포인트 앞선다. 차이는 결정 시점에 있다. CICS 는 하루 전에 시간별 곡선을 그리므로 개별 작업의 마감을 알지 못한 채 허용량을 배분하고, 곡선이 저탄소 시각에 자리를 열어도 그때까지 기다릴 수 있는 작업이 이미 남아 있지 않은 일이 생긴다. 본 연구의 알고리즘은 매 슬롯마다 각 작업의 남은 창을 다시 본다.

A.5 Sukprasert — 공개 아티팩트의 탐색 알고리즘

출처는 저자들이 공개한 아티팩트(zenodo.org/records/10790855)의 시뮬레이션 코드다. 시간 이동은 제출 시각부터 여유 시간까지의 창에서 연속 구간 합이 최소인 시작점을 고르고, 공간과 시간을 함께 쓰는 경우에는 리전과 시작 시각의 격자를 전수 탐색한다.

적용하며 내린 판단

원문 3 절 2 항이 "미래 탄소집약도와 작업 길이를 완전히 안다고 가정한다"고 밝히므로, 재현에서도 예측값이 아니라 실측 계열로 결정하였다. 원문의 워크로드 특성 표에 용량 항목이 없고 동시 실행 수를 확인하는 코드도 없어 용량 제약은 두지 않았다. 원문은 여유 시간을 24·168·480 시간 따위로 고정해 스윕하지만, 그 표가 여유 시간을 작업이 갖는 한 차원으로 일반화해 두었으므로 여기서는 작업마다 자기 마감을 그대로 썼다 — 고정 스윕값은 집계 분석을 위한 저자들의 선택이지 알고리즘의 제약이 아니다.

동점 처리는 두 경우를 구분해야 한다. 시간 축에서 같은 값이 여럿이면 가장 이른 시각을 택하는데, 이는 원문이 쓰는 pandas 의 최솟값 색인이 첫 항목을 돌려주는 것과 같은 동작이다. 반면 공간과 시간을 함께 풀 때 서로 다른 리전이 같은 값을 내는 경우의 규칙은 원문에 없다 — 원문은 목적지 리전을 하나 정해 두고 그 리전에 대해서만 계산하므로 그런 동점이 생기지 않는다. 본 재현에서는 리전 목록의 앞선 리전을 택했고, 이 선택은 본 연구가 정한 것이다. 다만 이 규칙이 결과를 만들지는 않는다 — 프랑스의 연평균 탄소집약도 13.6 은 전체 평균 279.8 의 20 분의 1 이라 다른 리전과 같은 값이 나오는 경우가 사실상 없고, 96.61%·99.75%라는 쏠림은 동점 처리가 아니라 이 격차에서 온다.

표에서 "시간 이동만"은 홈 리전 고정이 아니다. 공간 배정은 본 연구의 무릎점 자동 선택 결과를 그대로 물려받고 시간 축의 알고리즘만 Sukprasert 의 것으로 갈아끼운 대조다 — 그래서 평균 네트워크 지연 36.2 ms 가 본편 4.2 절의 무릎점 값과 같다. 공간 축까지 함께 푸는 것은 다음 행("공간 + 시간")이다.

표 S10. Sukprasert 알고리즘을 본 연구 데이터에 적용한 결과

정책 (완전예지, 용량 무제약)	총배출(kg)	절감률	평균 지연	용량 위반
시간 이동만	9,583.6	67.21%	36.2 ms	49,356
공간 + 시간	990.8	96.61%	120.4 ms	120,186

공간과 시간을 함께 쓴 경우의 96.61%는 알고리즘의 성취라기보다 데이터의 성질이다. 본 연구의 8 개 리전 가운데 프랑스의 연평균 탄소집약도는 13.6 gCO_2/kWh 로 전체 평균 279.8 의 약 20 분의 1 이다. 제약이 없으면 최적해는 사실상 "전부 프랑스로"가 되며, 실제로 146,000 건의 99.75%인 145,641 건이 프랑스에 배정되고 여덟 리전 중 여섯 곳은 한 건도 받지 못했다.

같은 현상을 원문도 보고한다. 원문 5 절 1 항 1 목은 "모든 작업이 스웨덴으로 이동한다"고 적으며 그 결과를 96% 감축으로 제시한다. 서로 다른 리전 집합과 서로 다른 연도의 자료에서 같은 구조가 같은 방식으로 나타났다는 점이 재현이 옳다는 근거가 된다.

다만 이 배치는 산술적으로만 성립한다. 같은 스케줄을 본 연구의 용량 상한으로 다시 세면 120,186 건이 위반이다. 원문 역시 이 값을 달성 가능한 결과가 아니라 이상적 상한으로 다루며, 5 절 1 항 2 목에서 전 리전이 50% 가동률일 때 감축이 51.5%로 떨어져 이상적 조건 대비 1.9 배 줄어든다고 보고한다. 그 용량 정의는 전역적으로 균일한 정적 가정이고 공간 축에만 적용되므로, 본 연구가 슬롯마다 강제하는 동적

상한과 같은 눈금에 놓을 수 없다. 두 연구가 같은 눈금에서 만나는 지점은 숫자가 아니라 결론이다 — 제약 없는 최적해는 실행할 수 없다는 것.

A.6 Caspian — 원문 정수 계획 정식화

Caspian 은 공개 코드가 없어 논문의 수식이 유일한 원본이다. 아래는 원문 2 절의 정식화를 번호까지 그대로 옮긴 것이다.

정식화

$$Z1(x) = \Sigma_j \Sigma_t \left[\Sigma_i \Sigma_{t' \in \Delta^t_i} x^{t'}_{ij} \cdot r_i \right] / (C_j^t \cdot I_j^t \cdot \rho_j) \quad \dots (1)$$

탄소 항 — 자원 점유를 그 슬롯의 탄소집약도로 나눈다. 깨끗하고 여유 있는 클러스터에 놓을수록 커진다.

$$Z2(x) = \Sigma_i \Sigma_j \Sigma_t x^t_{ij} / \gamma(i,t), \quad \gamma(i,t) = \max(1, l_i + t - d_i) \quad \dots (2)$$

마감 항 — 마감을 넘긴 정도로 나눈다. 늦을수록 작아지므로 마감은 하드 제약이 아니라 목적함수의 페널티다.

$$Z3(x) = \Sigma_i \Sigma_j \Sigma_t x^t_{ij} / \lambda(i,t), \quad \lambda(i,t) = t + l_i \quad \dots (3)$$

완료 시각 항 — 늦게 끝날수록 작아진다. 같은 조건이면 일찍 끝내는 쪽을 고른다.

$$\max \; Z(x) = \omega^{-}1 \cdot Z1 + \omega^{-}2 \cdot Z2 + \omega^{-}3 \cdot Z3, \quad \omega^{-}_k = \omega_k / \theta_k, \quad \theta_k = 1/Z^{-}_k \quad \dots (4)$$

$$\text{s.t.} \quad \Sigma_i \Sigma_{t' \in \Delta^t_i} x^{t'}_{ij} \cdot r_i \le C_j^t \quad \forall j,t \quad \dots (5) \quad \text{용량 — 하드}$$

$$\Sigma_j \Sigma_t x^t_{ij} \le 1 \quad \forall i \quad \dots (6) \quad \text{최대 1 회}$$

$$x^t_{ij} \le a_{ij} \quad \forall i,j,t \quad \dots (7) \quad \text{적합성}$$

$$x^t_{ij} \in \{0,1\} \quad \dots (8)$$

x^t_{ij} 는 작업 i 를 클러스터 j 에서 슬롯 t 에 시작하는지, r_i 는 필요 자원량, l_i 는 실행 시간(슬롯 수), d_i 는 마감, C_j^t 와 I_j^t 는 슬롯 t 의 가용 자원과 탄소집약도, ρ_j 는 전력 효율이다. Δ^t_i = {t−l_i+1, …, t} 는 슬롯 t 를 점유하게 되는 시작 시각의 집합이다. 마감은 식 (2)의 페널티로만 들어가고, 용량은 식 (5)의 하드 제약이다 — 본 연구와 정확히 반대다.

원문은 이 정식화를 완전한 정수 계획으로 풀지 않는다. Algorithm 2 가 각 Z_k 를 따로 LP 완화로 풀어 정규화 상수 θ_k 를 얻고, 결합 목적함수를 한 번 더 LP 로 풀어 분수해 $\bar{x}$ 를 구한 뒤, $b_i = \Sigma \bar{x}$ 로 우선순위를 매겨 Algorithm 3 의 그리디 배정으로 정수해를 만든다. 즉 LP 완화 + 그리디이며, 이는 원문의 설계이지 본 연구의 단순화가 아니다.

우리 데이터로 옮기며 내린 판단

표 S11. Caspian 정식화를 본 연구 데이터에 대응시킨 내역

항목	값	근거
N · M · T · δ	146,000 · 8 · 24 · 1 시간	본 연구 예측 지평·슬롯과 통일
a_{ij}	1 (전 리전 허용)	본 연구 기본 설정과 통일
$r_i \cdot C_j$^t	$1 \cdot K_r = 12$	본 연구 용량이 동시 실행 수라 식 (5)가 식 (3)과 같아짐
ρ_j	1	리전별 전력 효율 자료 없음. 원문도 실측이 아닌 가정
I_j^t	LSTM 예측	예측으로 결정하고 실측으로 채점하는 원칙과 통일
ω	원문 5 절이 시험한 값	임의로 고르면 재현 충실도를 공격받는다

경계 처리 규약은 한 번 실패하고 다시 설계했다. 처음에는 제출 후 48 시간이 지나도 미배정이면 강제 배치하도록 두었는데, 마감 여유가 수 분뿐인 작업까지 48 시간을 기다리게 되어 대기열이 계속 자랐다. 자기 마감을 한 시간 넘기면 강제 배치하는 것으로 바꾼 뒤 대기열이 안정되었다.

슬롯 해상도가 만드는 한계

Caspian 의 정식화는 슬롯보다 짧은 작업을 표현하지 못한다. l_i 가 슬롯 수이므로 6 초짜리 작업도 한 슬롯을 통째로 차지한다. 본 연구의 워크로드는 이 한계에 정면으로 걸린다.

표 S12. 슬롯 올림이 만드는 점유 부풀림 — k=3 이하가 전체의 60.1%다

지연 등급	건수	평균 실행 시간	올림 배율
k=1	43,830	2.03 시간	1.23 배
k=2	14,458	0.72 시간	1.81 배
k=3	14,716	4.1 분	14.68 배

지연 등급	건수	평균 실행 시간	올림 배율
k=4	29,244	38 초	93.80 배
k=5	43,752	6 초	598.58 배

이 부풀림이 결과를 얼마나 좌우하는지 보려고, 자원 요구량을 r_i = min(실행 시간, 1) 로 완화한 대조 실행을 같은 조건에서 돌렸다. 원문의 r_i 는 개수가 아니라 자원량이므로 이 읽기가 오히려 원문에 가깝다.

표 S13. 올림과 분수 점유의 비교 — 60 일 부분집합, 24,000 건, ω1=1

지표	올림 (r_i=1)	분수 (r_i=min(d,1))	변화
총 배출량	4,357.0 kg	3,588.0 kg	−17.6%
마감 위반	4,186	2,552	−39.0%
용량 위반	1,623	615	−62.1%

다만 무엇이 완화되고 무엇이 남는지는 구분해야 한다. 상한에 묶인 (리전, 시각)의 수는 두 모드가 거의 같다(1,134 대 1,136). l_i 곧 몇 슬롯을 점유하는가는 두 모드 모두 올림값을 쓰기 때문이다. 분수 점유가 바꾸는 것은 묶이는 횟수가 아니라 묶였을 때 몇 건이 거절되는가다. 슬롯 개수의 올림 자체는 이 정식화의 구조적 한계이며 r_i 를 분수로 두어도 사라지지 않는다.

그 묶임이 실제 점유 때문인지도 따로 쟀다. 올림 모델에서 상한에 도달한 1,134 건 가운데 실제 실행 구간으로 다시 세면 상한 미만이었던 것이 1,068 건, 곧 94.2%다. 슬롯 올림이 용량을 그만큼 헛되이 압박하고 있었다는 뜻이다.

인과율 버그와 그 영향

재현 과정에서 본 연구가 낸 버그를 기록해 둔다. 배치 시각을 제출 시각의 내림값으로 기록해 실제 제출보다 최대 한 시간 이를 수 있었고, 채점기가 "제출 전에 시작해 일찍 끝난 것"으로 계산해 마감 위반이 인위적으로 사라졌다. max(정수 시, 제출 시각)으로 고치자 같은 조건에서 용량 위반이 16,101 건에서 1,623 건으로 줄었다. 버그가 있을 때 "헛된 묶임" 비율은 0.0%로 측정됐고 고친 뒤 94.2%가 나왔다 — 측정값이 0 이라는 것이 현상이 없다는 뜻은 아니다.

이 버그를 찾은 경로는 전수 코드 검증이 아니라 이상 탐지였다(마감 위반 0 건이 원문 설계와 반대였다). 그럴듯하게 나쁜 쪽으로 틀린 결함이었다면 같은 경로로는 못 잡았을 것이다. 나머지 세 재현은 원문 수식·상수의 한 줄 대조(A.3~A.5), 원문이 보고한 현상과 같은 방향인지 확인, 같은 채점 함수 통과 확인(A.2)으로 검증했으나 이상 탐지를 거치지 않은 것은 사실이다.

본편 표 4 에 실은 값

146,000 건 전체를 두 모드로 돌렸다. 본편 표 4 에는 분수 배정 쪽을 싣는다 — 원문의 정식화에 더 가깝고 네 지표 모두 올림보다 낫다. 이런 대조를 Caspian 에만 둔 것은 다른 셋에는 대응하는 선택지가 없기 때문이다. CASPER 와 CICS 는 슬롯별 자원량을 연속값으로 풀고 Sukprasert 는 점유를 아예 세지 않아, 슬롯 점유를 올릴지 말지라는 물음 자체가 생기지 않는다.

표 S14. Caspian 전체 스케일 — 슬롯 점유를 올림한 경우와 분수로 둔 경우

지표	올림 (A.6 대조)	분수 (본편 표 4)
총배출	25,145.9 kg	20,308.4 kg
절감률	13.96%	30.51%
평균 지연	86.1 ms	105.9 ms
마감 위반	29,108 건	17,054 건
용량 위반	5,172 건	3,745 건

60 일 표본에서 본 방향(분수가 네 지표 모두 개선)이 전체 스케일에서도 그대로 나왔다. 다만 절감률의 절대값은 표본과 크게 다르다 — 60 일 표본은 큐가 아직 정상 상태에 이르지 않은 구간을 포함한다. 두 모드 모두 마감 위반이 만 건 단위로 나오는데, 이는 구현의 결함이 아니라 Caspian 이 마감을 목적함수의 페널티로 두는 설계에서 온다. 본 연구가 마감을 하드 제약으로 두고 용량을 양보하는 것과 정확히 반대다.

A.7 대응시키지 않은 정책과 그 이유

CarbonFlex 는 작업 하나가 쓰는 서버 대수를 늘리고 줄이는 탄력적 확장을 기제로 삼는다. 본 연구의 작업은 원자적이어서 리전과 시각만 바뀌며 그러한 다이얼이 없으므로, 대응시킬 수 있는 실행 자체가 존재하지 않는다.

GSLB 는 원문에서 탄소집약도와 거리의 가중합으로 점수를 매기며 가중치를 0.67 로 둔다. 고정 가중치를 쓴다는 점은 CASPER 와 같으나 값이 다르고 절충 대상도 다르므로, 두 시스템을 하나의 행으로 묶지 않았다.

A.8 선행 연구의 평가 규모

본 연구는 실제 클러스터 배포가 아닌 시뮬레이션 평가다. 비교 대상들의 평가 방식을 원문에서 확인한 결과는 다음과 같다.

표 S15. 선행 연구와 본 연구의 평가 방식·규모

연구	평가 방식	규모	기간
CICS	실제 운영 배포	사업자 데이터센터	—
CarbonFlex	클라우드 프로토타입	CPU·GPU 클러스터	—
CASPER	원문 표현 "simulation"	웹 서비스 모사	24 시간
Caspian	컨테이너 기반 4 클러스터	작업 200 건	24 시간
Sukprasert	추적 기반 시뮬레이션	123 개 리전	3 년
본 연구	추적 기반 시뮬레이션	8 개 리전 · 작업 146,000 건	1 년

탄소집약도는 계절에 따른 구조를 갖는다. 여름의 태양광과 겨울의 풍력, 리전 간 위상차는 하루 단위 실험으로는 관측할 수 없다. 본편 4.3 절이 보고하는 "용량을 두지 않았을 때 시간 이동이 만드는 절감의 71.9%가 한 리전에서 발생하고, 그 리전에서 상한을 넘긴 시간의 91.38%가 UTC 19~21 시, 곧 현지 한낮 세 시간에 몰린다"는 결과는 1 년치 자료 없이는 얻을 수 없다.

반면 실제 배포만 확인할 수 있는 것도 있다. 스케줄러가 클러스터 관리 계층의 실제 인터페이스와 맞물리는 비용, 작업 이관의 부대 비용, 초기 기동 지연이 그것이다. 본 연구는 이를 검증하지 못했으며 본편 5 절에 한계로 밝혔다.

A.9 지연 등급 분포를 바꾸어 재현한 결과

본편의 모든 수치는 지연 등급 분포 하나 위에 서 있다. 그 분포는 실측 트레이스가 아니라 설정한 값이므로, 분포를 바꾸면 결론이 어떻게 달라지는지 따로 재었다. 분포마다 작업 146,000 건을 같은 시드로 다시 만들고 공간 이동부터 전체 파이프라인을 처음부터 다시 돌렸다 — 시간 이동만 다시 돌리면 공간 배정이 옛 분포의 것이라 뜻이 없다. 기준도 분포마다 다시 계산했다.

표 S16. 지연 등급 분포를 바꾸어 전체 파이프라인을 다시 돌린 결과

분포 (k=1..5)	유연 비중	기준 (kg)	CAST 절감률	용량 위반	마감 위반	용량 비용	예측 비용	배율
1:1:2:2:4	19.93%	12,243.3	64.92%	0	0	0.23%p	0.98%p	0.23 배
2:2:2:2:2	39.92%	23,914.7	63.42%	173	0	1.52%p	0.62%p	2.45 배
3:1:1:2:3 (본편)	39.92%	29,225.6	63.03%	227	0	2.90%p	0.61%p	4.76 배
3:2:1:1:3	49.96%	32,311.4	62.31%	631	0	2.95%p	0.65%p	4.53 배
4:3:1:1:1	70.08%	43,912.3	58.30%	3,643	0	3.85%p	0.82%p	4.72 배
5:4:1:0:0	90.10%	55,654.8	58.82%	28,828	0	4.41%p	0.72%p	6.14 배

세 가지가 드러난다. 첫째, 마감 위반은 여섯 지점 전부에서 0 이다. 용량 위반이 0 에서 28,828 건까지 치솟는 동안에도 한 건도 나지 않았고, 슬롯 재배치가 가장 공격적인 조건에서 완전예지 오라클로 다시 돌려도 같았다 — 본편 3.4 절의 마감 보장이 부하와 무관하다는 것이 실측으로 확인된다.

둘째, 용량 위반은 유연 작업 비중에 초선형으로 늘어난다. 비중이 2.3 배가 되는 동안 (39.92% → 90.10%) 위반은 167 배로 늘어난다(173 건 → 28,828 건). 두 배율을 같은 구간에서 재려고 19.93% 지점은 뺐다 — 그 지점의 위반이 0 이라 비가 정의되지 않는다. 옮길 수 있는 작업이 많아질수록 모두가 같은 저탄소 구간을 노리기 때문이며, 본편의 중심 주장이 가리키는 방향과 같다.

셋째, 용량 비용과 예측 비용의 배율은 상수가 아니다. 0.23 배에서 6.14 배까지 벌어지고, 유연 비중이 약 27% 아래로 내려가면 순서가 뒤집혀 예측 비용이 더 커진다. 본편의 워크로드는 그 경계보다 13%포인트 위에 있다. 다만 이 경계는 위 다섯 가중치 경로를 따라갔을 때의 근사이지 유연 비중만으로 정해지는 값이

아니다 — 같은 39.92%라도 균등 분포에서는 용량 비용이 1.52%p 로 본편(2.90%p)의 절반이다. 등급 1 과 2 사이의 비율도 함께 작용한다.

A.10 아티팩트 — 코드와 재현 절차

본편의 모든 수치는 아래 절차로 다시 만들 수 있다. 선행 정책 재현 코드는 기존 파이프라인을 고치지 않고 별도 모듈로 두었다. 정수 계획은 CBC 를 스레드 지정 없이 (곧 단일 스레드로) 호출하고 난수 시드를 주입하지 않으므로 같은 입력에서 같은 해를 낸다 — 연속 4 주(672 슬롯, 11,200 건)를 서로 독립된 두 프로세스로 풀어 총배출이 소수점 여섯 자리까지 같음을 확인하였다.

구성

표 S17. 재현에 쓰이는 파일

파일	내용
scheduler/prior_harness.py	공용 채점 함수. 모든 재현이 여기를 통과한다
scheduler/prior_casper.py	CASPER MILP (A.3)
scheduler/prior_cics.py	CICS VCC (A.4)
scheduler/prior_sukprasert.py	Sukprasert 탐색 (A.5)
scheduler/prior_caspian.py	Caspian 정수 계획 (A.6)
scheduler/reproduce.py	본편 표·그림 재생성
paper/diagram/gen_*.py	그림 생성

실행

본편 표 1~3 과 그림은 한 명령으로 나온다.

```
python -m scheduler.reproduce
```

표 4 의 선행 정책은 정책마다 따로 돌린다. 어느 것이든 결과는 같은 채점 함수를 통과하므로 서로 비교할 수 있다.

```
python -m scheduler.prior_harness            # 채점 함수 자체 검증
```

```
python -m scheduler.prior_casper             # α=0.9 와 0.5 둘 다
python -m scheduler.prior_casper --alpha 0.9 # 하나만
python -m scheduler.prior_cics               # λp/λe = 0
python -m scheduler.prior_cics --sweep       # A.4 가중치 스윕
python -m scheduler.prior_sukprasert         # 시간 단독·공간+시간
python -m scheduler.run_caspian_sweep 1.0    # Caspian, ω1 지정
python -m scheduler.run_caspian_sweep 1.0 fractional
```

Caspian 만 모듈이 아니라 별도 드라이버로 돈다. prior_caspian.py 는 결정만 돌려주고 채점하지 않으므로 직접 실행할 진입점이 없다 — 위 드라이버가 그 결정을 받아 공용 채점 함수에 넘긴다. 첫 인자는 원문의 가중치 ω1 이고, 둘째 인자를 주면 정수 올림 대신 분수 배정을 쓴다.

A.1 의 정확 적분 재채점 표 두 개는 다음으로 만든다.

```
python -m scheduler.accounting_check_table5
```

입력 자료

표 S18. 저장소에 포함된 입력 자료

경로	내용
load_balancer/data/jobs.csv	작업 146,000 건의 명세 (id·제출 시각·출발 리전·등급·실행 시간)
load_balancer/data/latency_8x8.csv	리전 간 왕복 지연 행렬 (ms)
load_balancer/data/lstm_eval/	리전별 eval_records — 실측(y_true)과 예측(y_pred)
scheduler/data/	용량 스윕·민감도·오라클 실행의 저장된 산출물

탄소 회계에 쓰는 실측 계열과 배치 결정에 쓰는 예측 계열은 모두 eval_records 에서 읽는다 — 재현에 LSTM 추론을 다시 돌릴 필요가 없다는 뜻이다. 저장된 체크포인트에는 가중치만 남아 있어 학습률·에폭·최적화기는 복원하지 못했다. 모델을 처음부터 다시 학습시키려는 사람에게는 제약이지만, 예측 계열이 이미 파일로 있으므로 본 논문의 결과를 재현하는 데는 걸림돌이 되지 않는다.

기대 결과와 허용 오차

기준 시나리오 29,225.6 kg 과 제안 기법 10,805.0 kg(63.03%, 상한 위반 227 건)이 그대로 재현되어야 한다. 정수 계획 솔버가 동률을 처리하는 방식이 실행마다 달라질 수 있어 총 배출량이 최대 0.14% 흔들린다. 그보다 큰 차이가 나면 입력 자료나 회계가 어긋난 것이다.

필요한 것

Python 3.11 이상(개발과 검증은 3.14.7 에서 수행), PuLP 3.3.2 와 함께 배포되는 CBC 2.10.3, NumPy. 0.14% 의 흔들림은 솔버의 동률 처리에서 오므로, 같은 값을 그대로 얻으려면 솔버 버전을 맞추는 편이 낫다. 전체 재현에 보통 컴퓨터로 수십 분이 걸리며, A.4 의 가중치 스윕과 A.6 의 Caspian 재현은 그보다 오래 걸린다.